

Three Address Code(TAC):

Aim:

To write a simple C program to generate Three Address Code (TAC) for an arithmetic expression such as $a = b + c * d$.

Algorithm:

1. ☐ Start the program.
2. ☐ Read the expression.
3. ☐ Store operands and operators in separate stacks.
4. ☐ Check operator precedence.
5. ☐ Generate temporary variables $t_1, t_2, \dots$
6. ☐ Generate Three Address Code.
7. ☐ Assign the final result to the target variable.
8. ☐ Stop the program.

Pseudocode:

START

Read expression

Separate target and RHS

Scan RHS from left to right

If operand $\rightarrow$ push into operand stack

If operator $\rightarrow$ push into operator stack

If higher precedence operator $\rightarrow$ process first

Generate:

$t_1 = \text{operand operator operand}$

Repeat until all operators are processed

Assign final result to target

STOP

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

#define MAX 100

char opStack[MAX];
int opTop = -1;

char *valStack[MAX];
int valTop = -1;

int tempCount = 1;

void pushOp(char ch)
{
    if (opTop < MAX - 1)
        opStack[++opTop] = ch;
}

char popOp()
{
    if (opTop >= 0)
        return opStack[opTop--];

    return '\0';
}

char peekOp()
{
    if (opTop >= 0)
        return opStack[opTop];

    return '\0';
}
```

```

void pushVal(char *val)
{
    if (valTop < MAX - 1)
        valStack[++valTop] = strdup(val);
}

```

```

char *popVal()
{
    if (valTop >= 0)
        return valStack[valTop--];

    return NULL;
}

```

```

int precedence(char op)
{
    if (op == '+' || op == '-')
        return 1;

    if (op == '*' || op == '/')
        return 2;

    return 0;
}

```

```

void processOperator()
{
    char op = popOp();

    char *right = popVal();
    char *left = popVal();

    char *temp = (char *)malloc(10 * sizeof(char));

    sprintf(temp, "t%d", tempCount++);

    printf("%s = %s %c %s\n",
        temp, left, op, right);
}

```

```

    pushVal(temp);

    free(left);
    free(right);
}

void generateTAC(char *expr)
{
    int i = 0;

    char target[MAX] = "";
    int targetIdx = 0;

    /* Separate target variable from RHS */
    while (expr[i] != '=' && expr[i] != '\0')
    {
        if (!isspace(expr[i]))
            target[targetIdx++] = expr[i];

        i++;
    }

    target[targetIdx] = '\0';

    if (expr[i] == '=')
    {
        i++;
    }
    else
    {
        i = 0;
        target[0] = '\0';
    }

    printf("\nThree Address Code (TAC):\n");
    printf("-----\n");

```

```

/* Process RHS */
while (expr[i] != '\0')
{
    if (isspace(expr[i]))
    {
        i++;
        continue;
    }

    /* Operand */
    if (isalnum(expr[i]))
    {
        char operand[MAX];
        int k = 0;

        while (isalnum(expr[i]))
        {
            operand[k++] = expr[i++];
        }

        operand[k] = '\0';

        pushVal(operand);
    }

    /* Opening bracket */
    else if (expr[i] == '(')
    {
        pushOp(expr[i]);
        i++;
    }

    /* Closing bracket */
    else if (expr[i] == ')')
    {
        while (opTop >= 0 && peekOp() != '(')
        {
            processOperator();
        }
    }
}

```

```

    }

    popOp();
    i++;
}

/* Operator */
else if (expr[i] == '+' ||
        expr[i] == '-' ||
        expr[i] == '*' ||
        expr[i] == '/')
{
    while (opTop >= 0 &&
           precedence(peekOp()) >= precedence(expr[i]))
    {
        processOperator();
    }

    pushOp(expr[i]);
    i++;
}

else
{
    i++;
}
}

/* Process remaining operators */
while (opTop >= 0)
{
    processOperator();
}

/* Assign final result */
char *finalRes = popVal();

if (target[0] != '\0' && finalRes != NULL)

```

```
{
    printf("%s = %s\n", target, finalRes);
}

if (finalRes)
    free(finalRes);
}

int main()
{
    char expr[MAX] = "a = b * c + d";

    printf("Input Expression: %s\n", expr);

    generateTAC(expr);

    return 0;
}
```

OUTPUT:

Output

Input Expression: $a = b * c + d$

Three Address Code (TAC):

$t1 = b * c$

$t2 = t1 + d$

$a = t2$